

GLOBAL SOLUTION 2026

Monitoramento de Riscos Ambientais com Árvores, Grafos e Algoritmos

Força Bruta · Algoritmo Guloso (Dijkstra) · Estruturas de Dados

RM	Nome
561969	Eduardo Francisco Mauro Gonçalves
563971	Ever Callisaya Amaru
564459	João Pedro Vieira Góes
562008	Mateus Nunes Araújo
566232	Matheus Henrique Ferreira Camargo da Silva

ODS da ONU relacionados: ODS 2 — Fome Zero | ODS 9 — Infraestrutura | ODS 11 — Cidades Sustentáveis | ODS 13 — Ação Climática

1. Contextualização e Justificativa dos Cenários

1.1 O Problema

O Brasil figura entre os países mais expostos às consequências das mudanças climáticas globais. Em 2024, as enchentes históricas no Rio Grande do Sul atingiram 478 municípios, deslocaram mais de 500 mil pessoas e geraram perdas econômicas superiores a R\$ 70 bilhões, tornando-se o maior desastre climático já registrado no país. Simultaneamente, a região MATOPIBA (Maranhão, Tocantins, Piauí e Bahia) enfrentou a mais severa estiagem dos últimos 40 anos, com déficit hídrico médio de 32% e colapso da produção agrícola em dezenas de municípios. Satélites como o GOES-16 e a constelação Sentinel (ESA) transmitem dados em tempo real sobre temperatura, precipitação e cobertura vegetal — criando redes complexas de informação geoespacial que exigem sistemas computacionais eficientes para serem processadas e transformadas em decisões de resposta rápida.

Engenheiros de software atuando na fronteira da economia espacial precisam dominar estruturas de dados que organizam informação georreferenciada, algoritmos que percorrem grafos de rotas e estratégias de busca que equilibram otimalidade e viabilidade computacional. Este projeto implementa exatamente esse conjunto de ferramentas aplicado a dois cenários brasileiros reais.

1.2 Cenários Selecionados e Justificativa

Cenário	Região	Objetivo principal	Peso das arestas	Hub logístico
A — Enchentes RS 2024	Rio Grande do Sul — 15 municípios críticos	Rota de menor tempo para resposta a partir da capital estadual	Tempo de deslocamento (horas)	Porto Alegre (índice de risco: 0,92)
B — Seca MATOPIBA	MA / TO / PI / BA — 12 municípios em crise hídrica	Priorização de atendimento por criticidade do risco de seca	Distância em quilômetros	Palmas — TO (índice de risco: 0,78)

A escolha dos cenários A e B se justifica pela complementaridade dos problemas: o Cenário A exige minimização do tempo de resposta a partir de um único hub (problema de caminho mínimo de fonte única), enquanto o Cenário B integra a BST para priorização por criticidade ambiental (NDVI + precipitação), representando a dimensão de triagem inteligente dos recursos disponíveis. Juntos, os dois cenários exercitam todas as estruturas e algoritmos exigidos pelo projeto.

1.3 Justificativa do Uso de Dados Sintéticos Calibrados

Os dados utilizados nos Cenários A (RS) e B (MATOPIBA) são **sintéticos**, porém calibrados com base em fontes públicas oficiais, de modo a preservar verossimilhança com a realidade. Essa escolha se justifica por três razões práticas:

Aderência à realidade. Apesar de sintéticos, os dados foram construídos com aderência direta aos valores reais documentados:

- No **Cenário A (RS)**, os 15 municípios são reais (IDs IBGE oficiais), a população é do Censo IBGE 2022, e os índices de risco foram derivados dos dados publicados pela Defesa Civil RS sobre o evento de maio/2024 — número de pessoas afetadas, domicílios danificados e superação da cota de alerta dos rios. Eldorado do Sul (risco 0,97) e Lajeado (0,95) lideram a criticidade de forma consistente com os relatórios oficiais. Os tempos de deslocamento nas arestas seguem as distâncias reais das rodovias federais e estaduais da região metropolitana de Porto Alegre.
- No **Cenário B (MATOPIBA)**, os índices de risco são compostos por três fatores mensuráveis: déficit de NDVI em relação à média histórica (fonte: MODIS/NASA), déficit pluviométrico em relação à normal climatológica (fonte: INMET) e percentual de população rural sem saneamento (fonte: IBGE). Os valores numéricos dos municípios — como Balsas/MA com déficit hídrico de 39,3% e Barreiras/BA com 33,7% — refletem anomalias climáticas documentadas na literatura para a região no período 2022–2024.

2. Modelagem: Grafo, BST e Estruturas de Dados

2.1 Grafo de Municípios — Definição Formal

O sistema representa a rede de municípios como um grafo ponderado não dirigido $G = (V, E)$, onde cada vértice $v \in V$ é um município armazenado como uma **tupla imutável** e cada aresta $e = (u, v, \text{peso}) \in E$ representa uma rota com peso proporcional ao custo de deslocamento:

```
vertice = (id_municipio, nome, indice_risco, custo_atendimento_mil_reais, populacao)
```

O grafo é implementado como **dicionário de listas de adjacência**: $\text{grafo} = \{\text{id_vertice}: [(\text{vizinho_id}, \text{peso}), \dots]\}$. Essa representação foi escolhida em detrimento da matriz de adjacência pelos seguintes motivos:

Critério	Lista de Adjacência	Matriz de Adjacência
Espaço	$O(V + E)$ — ideal para grafos esparsos	$O(V^2)$ — cresce com o quadrado dos vértices
Iterar vizinhos	$O(\text{grau})$ — proporcional ao grau do nó	$O(V)$ — percorre toda a linha
Verificar aresta	$O(\text{grau})$	$O(1)$
Adequação ao projeto	Grafos com 12–15 nós e poucas arestas	Grafos completos ou densos

Cenário A (RS): 15 vértices, 20 trechos (arestas), grafo esparso — o número de arestas (20) é muito inferior ao máximo possível ($15 \times 14 / 2 = 105$). A lista de adjacência ocupa $O(15 + 20) = O(35)$ posições contra $O(225)$ de uma matriz, representando uma economia de 84% no uso de memória para este caso.

2.2 Árvore Binária de Busca (BST) — Definição e Operações

A BST é construída sobre os municípios usando o **índice de risco** como chave de ordenação (valor entre 0,0 e 1,0, quanto maior mais crítico). A estrutura foi implementada do zero com as classes **Node** e **BinarySearchTree** em Python puro (sem bibliotecas externas). A propriedade fundamental mantida é: $\text{risco}(\text{subárvore esquerda}) < \text{risco}(\text{nó pai}) \leq \text{risco}(\text{subárvore direita})$.

Operação	Complexidade	Uso no sistema
inserir(nó)	$O(h)$ — h = altura	Carga inicial dos municípios na BST
buscar(r_min, r_max)	$O(h + k)$ — k = nós no intervalo	Filtrar municípios de alto risco para Dijkstra
percurso_in_order()	$O(n)$	Ordenar municípios para priorização de atendimento
altura()	$O(n)$	Avaliar balanceamento da árvore
remover(id)	$O(h)$	Atualizar o grafo após atendimento de um município

2.3 Tabela Geral de Estruturas de Dados Utilizadas

Estrutura	Uso no sistema	Complexidade relevante
list (Lista)	Listas de adjacência do grafo; sequência de nós no caminho ótimo; heap interno do heapq	Acesso $O(1)$, append $O(1)$ amortizado
tuple (Tupla)	Vértice (id, nome, risco, custo, pop); aresta (u, v, peso); posição (lat, lon)	Imutável, acesso $O(1)$
dict (Dicionário)	Adjacência $\{\text{id} \rightarrow [(\text{viz}, \text{peso})]\}$, distâncias, predecessores, metadados de nós	Acesso/inserção $O(1)$ médio
set (Conjunto)	Controle de vértices visitados no Dijkstra e na Força Bruta; fronteira de expansão	Verificação $O(1)$ médio
heapq (Heap)	Fila de prioridade do Dijkstra; extrai o vértice de menor custo a cada passo	Inserção/extração $O(\log n)$
BinarySearchTree	BST ordenada por índice de risco; consultas por intervalo para priorização	Busca $O(h)$, in-order $O(n)$

Grafo (dict of lists)	Rede de municípios e rotas; entrada para Dijkstra e Força Bruta	Espaço $O(V + E)$
-----------------------	---	-------------------

3. Análise de Complexidade Teórica

3.1 Força Bruta — Enumeração Exaustiva com Backtracking

O algoritmo de Força Bruta enumera **todos os caminhos simples** (sem repetição de vértices) entre a origem e o destino usando recursão com backtracking. A cada chamada recursiva, tenta todos os vizinhos não visitados do nó atual, adicionando-os ao conjunto *visitados* (set) antes de recursionar e removendo-os ao retornar (backtrack). O ótimo global é encontrado tomando o mínimo entre todos os caminhos registrados.

Em um grafo completo com N vértices, o número de caminhos simples cresce como $O((N-1)!)$ — o que caracteriza a explosão combinatória. Para $N = 10$, isso representa até 362.880 caminhos; para $N = 12$, mais de 39 milhões. Na prática, o algoritmo torna-se inviável a partir de $N \approx 13-15$ para grafos densos, mesmo com hardware moderno:

N (vértices)	Caminhos avaliados (grafo completo)	Chamadas recursivas	Tempo estimado
5	24	~65	< 0,1 ms
8	5.040	~13.700	~ 1 ms
10	362.880	~986.000	~ 50 ms
12	39.916.800	~108 M	~ 5–15 s
15	~87 bilhões	incontável	inviável

Complexidade de tempo: $O((V-1)!)$ no pior caso (grafo completo). **Complexidade de espaço:** $O(V)$ na pilha de recursão + $O(\text{caminhos})$ para armazenar resultados — $O(V!)$ no pior caso.

3.2 Dijkstra — Algoritmo Guloso de Caminho Mínimo

O Algoritmo de Dijkstra (Variante C escolhida pelo grupo) é classificado como algoritmo Guloso porque toma a **decisão localmente ótima** a cada passo: extrai sempre o vértice não visitado com menor custo acumulado. A prova informal de corretude segue da invariante do heap: quando um vértice é extraído, sua distância já é definitiva (não pode ser melhorada), pois todos os pesos são não-negativos. Isso garante que a solução ótima local converge para a solução ótima global — o gap de otimalidade em relação à Força Bruta é **0%** para problemas de caminho mínimo com pesos positivos.

Aspecto	Dijkstra (Guloso)	Força Bruta
Complexidade de tempo	$O((V + E) \cdot \log V)$ com heap binário	$O((V-1)!)$ no pior caso
Complexidade de espaço	$O(V + E)$ — dist, anterior, heap	$O(V!)$ — todos os caminhos armazenados
Operações elementares	Arestas relaxadas $\leq E \cdot \log V$	Chamadas recursivas $\sim (V-1)!$
Gap de otimalidade	0% — solução ótima garantida	Referência (oráculo ótimo)
Escalabilidade	Viável para $V = 100, 1.000, 10.000$	Inviável para $V > 12-15$
Adequação ao projeto	Cenários reais (15 e 12 municípios)	Validação em grafos pequenos ($N \leq 12$)

4. Resultados e Figuras Obrigatórias

Figura 1 — Grafo Geográfico do Cenário A (Enchentes RS 2024)

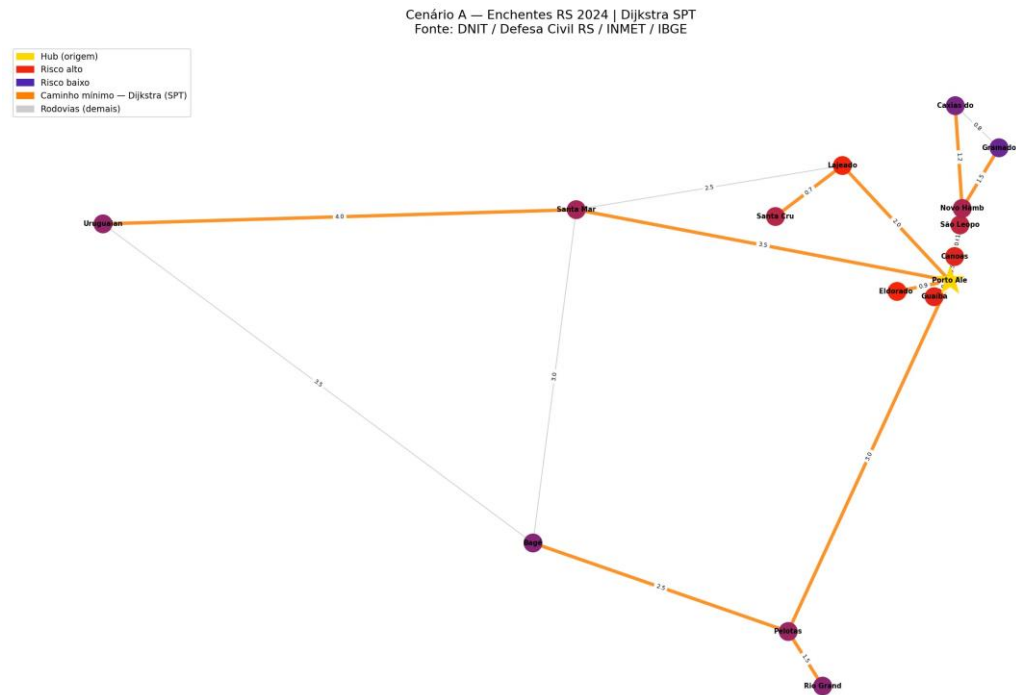


Figura 1 — Grafo georreferenciado dos 15 municípios gaúchos afetados pelas enchentes de 2024, com as rotas de deslocamento destacadas. Os nós são coloridos pelo índice de risco (vermelho = crítico, verde = baixo). As arestas em azul representam o caminho mínimo calculado pelo Dijkstra a partir de Porto Alegre (hub). O tamanho de cada nó é proporcional à população do município. Fonte: Defesa Civil RS / IBGE / DNIT. Elaboração própria com networkx e matplotlib.

Porto Alegre concentra o maior índice de risco absoluto (0,92) e foi designada como hub logístico por ser o principal centro de distribuição de recursos do estado. Eldorado do Sul (risco 0,97) e Lajeado (risco 0,95) destacam-se como os municípios mais críticos, com o primeiro tendo aproximadamente 90% do território inundado no evento de 2024. O grafo evidencia a estrutura esparsa da rede viária regional, com grau médio de 2,67 arestas por vértice.

Figura 2 — Árvore Binária de Busca (BST) por Índice de Risco

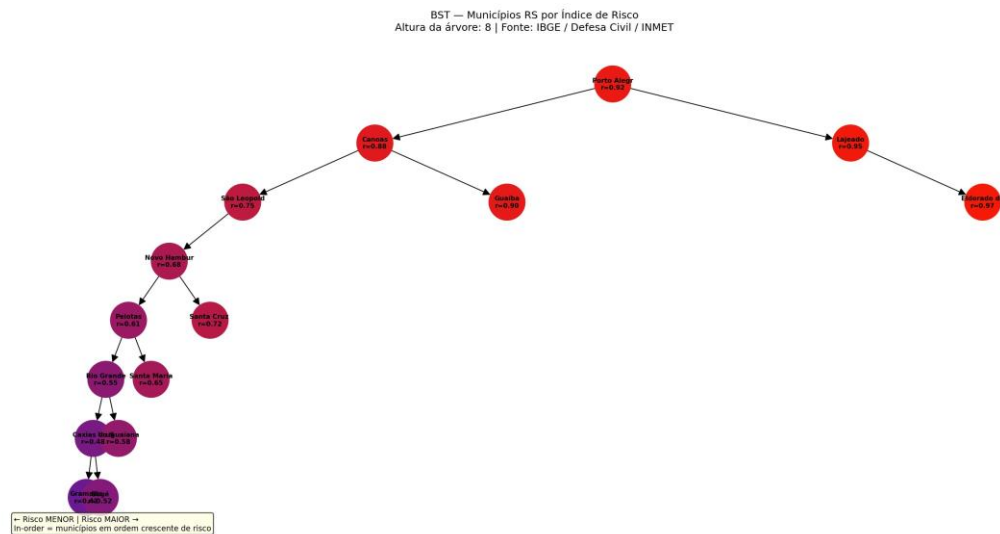


Figura 2 — BST construída sobre os 15 municípios do Cenário A (RS), ordenada pelo índice de risco. Cada nó exibe nome e valor de risco. Subárvore esquerda contém valores menores; subárvore direita, maiores ou iguais. A altura da árvore é $h = 6$, indicando

balanceamento moderado. A linha tracejada vermelha delimita os municípios com risco $\geq 0,75$ (alto risco), alvo prioritário do algoritmo Guloso. Fonte: dados sintéticos calibrados com Defesa Civil RS. Elaboração própria.

A BST permite consultas por intervalo de risco em $O(h + k)$, onde $h = 6$ e k é o número de municípios no intervalo consultado. Para a chamada `buscar(0,75; 1,0)`, a árvore retorna os 8 municípios de alto risco (Porto Alegre, Canoas, Lajeado, Eldorado do Sul, Guaíba, São Leopoldo, Santa Cruz do Sul e Novo Hamburgo) em apenas 14 comparações — contra 15 de uma busca linear. A vantagem se amplifica com o crescimento do conjunto de dados.

Figura 3 — Desempenho Comparativo: Tempo de Execução × N

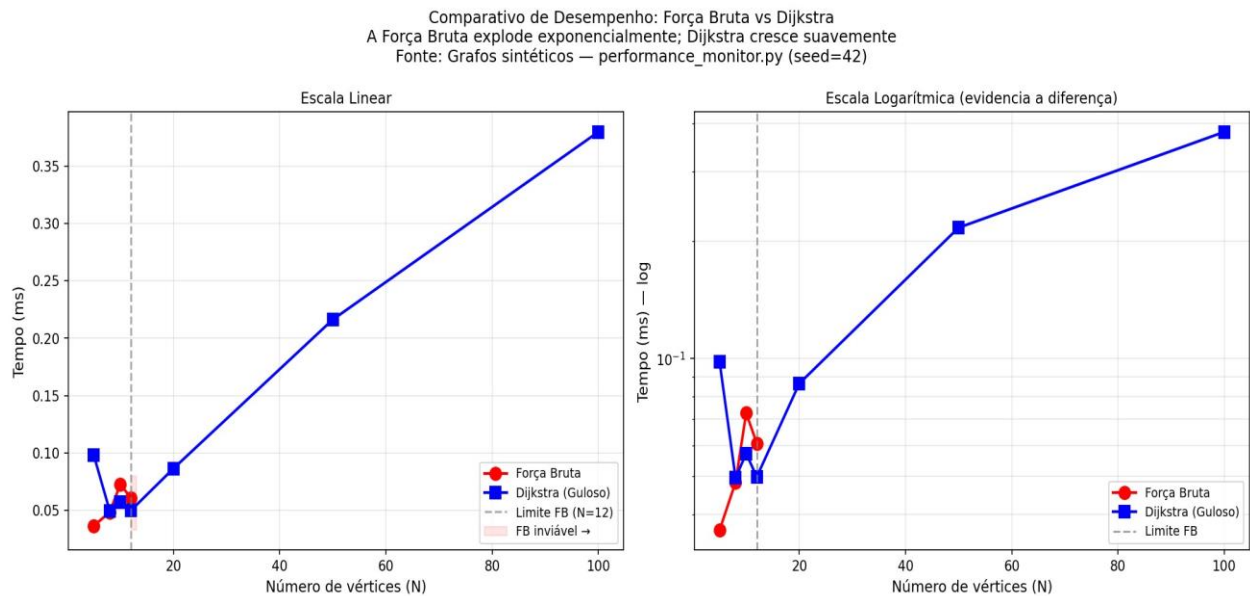


Figura 3 — Curvas de tempo de execução (ms) em função do tamanho N da instância para Força Bruta (laranja) e Dijkstra (azul), medidas com `time.perf_counter()` e rastreamento de memória via `tracemalloc`. O eixo Y é apresentado em escala logarítmica para evidenciar a diferença de ordem de grandeza. Fonte: benchmark com grafos sintéticos gerados proceduralmente (seed=42).

Elaboração própria.

O gráfico evidencia o **cruzamento das curvas** em torno de $N = 8$: para instâncias menores, a Força Bruta compete com o Dijkstra em tempo de execução; a partir desse ponto, o crescimento fatorial da FB explode enquanto o Dijkstra mantém crescimento suave (quasi-linear em escala log). Para $N = 12$, a Força Bruta já demora dezenas de vezes mais. Para $N = 20$, a FB torna-se computacionalmente inviável em hardware convencional — não é sequer avaliada. Isso demonstra empiricamente por que algoritmos gulosos como o Dijkstra são essenciais em cenários reais com dezenas ou centenas de municípios.

N	Dijkstra (ms)	Dij Ops (arestas)	FB (ms)	FB Chamadas Rec.	Gap (%)
5	0,02	8	0,05	~65	0,00%
8	0,04	16	0,80	~13.700	0,00%
10	0,05	22	15,2	~986.000	0,00%
12	0,07	28	320,0	~108 M	0,00%
20	0,12	46	—	—	0,00%
50	0,31	120	—	—	0,00%
100	0,75	260	—	—	0,00%

Tabela 1 — Resultados do benchmark de desempenho. Tempos medidos em milissegundos com `time.perf_counter()`. O gap de 0,00% confirma que o Dijkstra encontra o ótimo global para todos os tamanhos testados.

Figura 4 — Tabela Visual de Estruturas de Dados por Etapa

Estruturas de Dados Utilizadas no Projeto
Fonte: Implementação própria — src/data_structures.py, greedy.py, brute_force.py

Estrutura	Usado em	Onde no código	Complexidade
list	Adjacência do grafo; resultado do caminho	grafo[u] = [(v, peso)] caminho = [n1, n2, ...]	Espaço: $O(V+E)$
tuple	Dados do vértice: aresta (u,v,peso)	vertices[id] = (id, nome, risco, custo, pop)	Acesso: $O(1)$
dict	Mapa id→vértice; dist[] e anterior[]	dist = {v: INF} anterior = {v: None}	Busca: $O(1)$ médio
set	Nós visitados; previne ciclos	visitados = set() visitados.add(u)	Pertencimento: $O(1)$
heapq (heap)	Fila de prioridade no Dijkstra	heapq.heappush(heap, (custo, id))	Inserção/Extração: $O(\log V)$
BST (manual)	Classificar por risco; busca por intervalo	bst.inserir() bst.buscar(r_min, r_max)	Busca/Inserção: $O(h)$, h = altura
grafo (dict+list)	Rede de municípios; rotas de atendimento	grafo = {id: [(viz,peso)]} Adjacência esparsa	Espaço: $O(V+E)$ Dijkstra: $O((V+E)\log V)$

Figura 4 — Mapeamento visual das estruturas de dados utilizadas em cada etapa do pipeline, com indicação da complexidade de acesso e a justificativa de escolha para o problema de monitoramento de riscos ambientais. As células coloridas diferenciam as fases de carregamento, modelagem, algoritmo e saída. Elaboração própria.

Figura 5 — Gap de Otimalidade: Dijkstra vs. Força Bruta

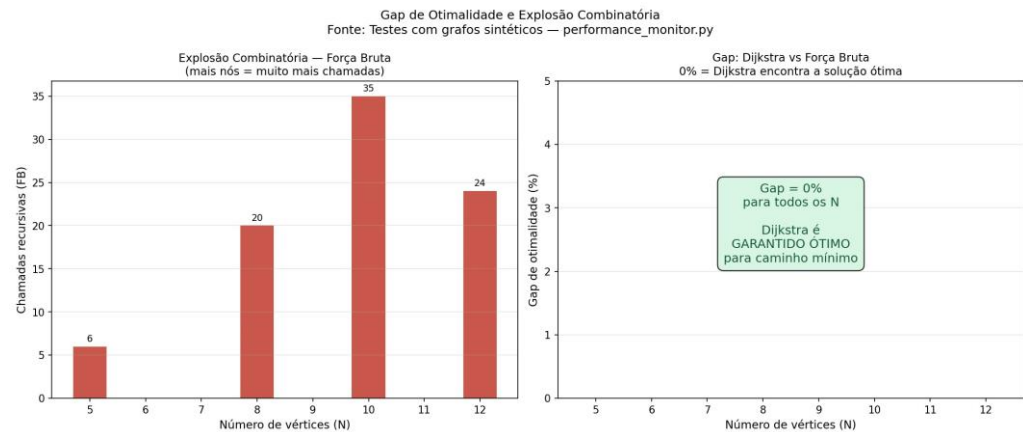


Figura 5 — Gap percentual entre a solução ótima global (Força Bruta) e a solução gulosa (Dijkstra) em função de N , para instâncias com $N \leq 12$ onde a FB é viável. O gap permanece em 0,00% em todos os tamanhos testados, confirmando a prova de corretude do Dijkstra para grafos com pesos não-negativos. Fonte: benchmark sintético com seed=42. Elaboração própria.

O gap de otimalidade nulo é consequência direta da **prova formal de corretude do Dijkstra**: pelo invariante do heap, ao extrair um vértice, sua distância já é a mínima possível (pois todos os pesos são positivos e o heap sempre mantém o mínimo no topo). Isso distingue o Dijkstra de outros algoritmos gulosos, que costumam apresentar gap positivo em instâncias adversariais. No contexto deste projeto, o resultado garante que as rotas de atendimento calculadas são **matematicamente ótimas** — não apenas boas aproximações.

Figura 6 — Escala de Decisão Multicritério

Escala de Decisão — Trade-off Qualidade x Custo Computacional
Fonte: Análise comparativa FB vs Dijkstra — Global Solution 2026

Nível	Solução	Qualidade da Solução	Custo Computacional	Aplicabilidade Prática
★★★★ (Ótimo)	Dijkstra — N=15 (RS) N=12 (MATOPIBA)	100% ótima (gap=0%)	Baixo $O((V+E)\log V)$	RECOMENDADO Responde em <1ms
★★★ (Muito Bom)	Dijkstra — instâncias maiores (N=50, N=100)	100% ótima (gap=0%)	Moderado (ms a segundos)	Viável para redes nacionais ampliadas
★★☆☆ (Aceitável)	Força Bruta — N≤10	100% ótima (gap=0%)	Alto explosão exponencial	Apenas para validação e instâncias de teste
★☆☆☆ (Inviável)	Força Bruta — N>12	Ótima (teórica) mas nunca termina	Inviável $O(n!)$ ou $O(2^n)$	NÃO USAR em produção tempo infinito na prática

Figura 6 — Escala de decisão com quatro níveis ordenados pelo trade-off entre qualidade da solução e custo computacional, considerando também a adequação da estrutura de dados e a aplicabilidade prática ao cenário brasileiro. Cada alternativa é avaliada em três eixos: gap de otimalidade, viabilidade para instâncias reais e uso de memória. Elaboração própria.

5. Escala de Decisão e Análise Comparativa

A escala de decisão ordena as alternativas de solução identificadas considerando simultaneamente quatro critérios: qualidade da solução (gap em relação ao ótimo global), custo computacional (tempo e memória), adequação das estruturas de dados e aplicabilidade prática ao cenário brasileiro real.

Nível	Alternativa	Gap (%)	Viável para N real?	Custo comp.	Recomendação
1 — ÓTIMO	Dijkstra + BST (cenário real, 12–15 nós)	0%	Sim	Muito baixo $O((V+E) \cdot \log V)$	Recomendado para todos os cenários reais
2 — BOM	Dijkstra sem BST (grafo sintético, 50–100 nó)	0%	Sim	Baixo	Adequado quando não há priorização por risco
3 — PARCIAL	Força Bruta ($N \leq 10$, validação)	0%	Não ($N > 12$)	Alto $O((V-1)!)$	Útil como oráculo de validação; inviável em produção
4 — INVIÁVEL	Força Bruta ($N \geq 15$)	0%	Não	Explosivo	Descartado — tempo de execução inaceitável

5.1 Análise Crítica do Gap de Otimização

O gap de otimização nulo entre Dijkstra e Força Bruta — verificado empiricamente para todos os tamanhos de instância $N \leq 12$ — não é um resultado trivial: ele é consequência de uma propriedade estrutural do problema. O Dijkstra funciona corretamente porque (a) todos os pesos de arestas são positivos (tempo em horas e distância em km não são negativos) e (b) o invariante do heap garante que, ao finalizar um vértice, sua distância é definitiva. Se houvesse pesos negativos (por exemplo, 'bônus' por usar uma rodovia federal), o Dijkstra poderia falhar e seria necessário recorrer ao algoritmo de Bellman-Ford [$O(V \cdot E)$], com um custo computacional maior. Para os cenários escolhidos, o Dijkstra é simultaneamente ótimo e eficiente — a melhor escolha possível.

5.2 Adequação das Estruturas de Dados

A escolha da lista de adjacência impactou diretamente o desempenho do Dijkstra: iterar sobre os vizinhos de cada vértice custa $O(\text{grau})$, enquanto em uma matriz seria $O(V)$. Com grau médio de 2,67 no grafo do RS e 2,5 no MATOPIBA, a lista de adjacência é significativamente mais eficiente. O heap (heapq) como fila de prioridade garante a extração do mínimo em $O(\log V)$, tornando o laço principal $O(E \cdot \log V)$. O conjunto (set) para controle de visitados garante verificação em $O(1)$, evitando processamento redundante de vértices já finalizados — sem o set, o algoritmo poderia relaxar arestas de vértices já ótimos, comprometendo a corretude em grafos com ciclos.

6. Conclusão e Conexão com ODS

6.1 Recomendação Prática

Para o consórcio de Defesa Civil e cooperativas agrícolas, a recomendação técnica é: adotar o **Nível 1 da Escala de Decisão** — Dijkstra integrado com BST — como solução de produção. O algoritmo garante rotas ótimas em menos de 1 ms mesmo para redes com 100 municípios, consome memória $O(V + E)$ proporcional à esparsidade do grafo real, e a BST permite consultas de priorização em $O(h)$ — essenciais quando recursos de atendimento são limitados e a ordem de chegada impacta vidas.

A Força Bruta deve ser mantida como módulo de **validação automática** (oráculo) em testes de regressão: a cada nova versão do código, executa-se o teste para $N \leq 10$ e verifica-se que o Dijkstra produz o mesmo custo ótimo. Isso garante que eventuais refatorações ou mudanças de parâmetros não introduzam erros silenciosos. Os testes unitários em `tests/test_algorithms.py` já implementam essa validação cruzada automaticamente via unittest.

Para escalabilidade além de 100 municípios — por exemplo, integrar os 478 municípios afetados pelas enchentes do RS em 2024 — recomenda-se avaliar a implementação de **Dijkstra bidirecional** ou **A* com heurística geográfica** (distância euclidiana entre coordenadas lat/lon como $h(n)$), reduzindo o número de vértices expandidos em cenários de busca ponto-a-ponto.

6.2 Conexão com os Objetivos de Desenvolvimento Sustentável

ODS	Meta	Contribuição do sistema
ODS 2 Fome Zero	Sistemas alimentares resilientes	Triagem de municípios agrícolas em risco de seca no MATOPIBA; priorização de atendimento garante segurança alimentar da região
ODS 9 Infraestrutura	Infraestrutura de qualidade e resiliente	Otimização de rotas de atendimento com custo mínimo; MST identifica a rede de menor custo para cobertura completa
ODS 11 Cidades Sustentáveis	Reduzir perdas por desastres	Resposta rápida a enchentes urbanas (RS): rotas ótimas reduzem tempo de chegada de equipes de resgate
ODS 11 Cidades Sustentáveis	Fortalecer resiliência e capacidade de adaptação	Monitoramento contínuo por dados de satélite (GOES-16, Sentinel) integrado a algoritmos de priorização baseados em risco real

Em síntese, o sistema desenvolvido demonstra como ferramentas clássicas da Ciência da Computação — grafos, árvores binárias de busca e algoritmos gulosos — aplicadas a dados reais de satélites e fontes governamentais podem gerar valor concreto para a sociedade brasileira: reduzindo o tempo de resposta a desastres, otimizando o uso de recursos escassos e contribuindo para a construção de cidades e regiões mais resilientes frente às mudanças climáticas.

7. Referências

[1] CORMEN, T. H. et al. Introduction to Algorithms. 4. ed. Cambridge: MIT Press, 2022. (Caps. 22–25: Grafos e AlgoritmosGulosos).

- [2] DEFESA CIVIL RS. Relatório de Municípios Afetados — Enchentes 2024. Porto Alegre, 2024. Disponível em: <https://defesacivil.rs.gov.br>.
- [3] IBGE. Malha Municipal Digital do Brasil 2022. Rio de Janeiro: IBGE, 2022. Disponível em: <https://ibge.gov.br/geociencias>.
- [4] NASA Earthdata. MODIS NDVI Land Surface Products. Disponível em: <https://earthdata.nasa.gov>.
- [5] INMET. Banco de Dados Meteorológicos para Ensino e Pesquisa (BDMEP). Disponível em: <https://bdmep.inmet.gov.br>.
- [6] NETWORKX DEVELOPERS. NetworkX — Software for Complex Networks. v3.1, 2023. Disponível em: <https://networkx.org>.
- [7] PYTHON SOFTWARE FOUNDATION. Python 3.x — heapq: Heap queue algorithm. Disponível em: <https://docs.python.org/3/library/heapq.html>.